

Homework 3

GWU CSCI 1112 - Spring 2026

February 25, 2026

1 Introduction

This homework is intended to reinforce your understanding of multidimensional arrays, sorting algorithms, recursion, implicit algorithm costs, and performing efficiency analysis. Your task is to implement and analyze the cost of different algorithmic approaches using the provided framework.

1.1 Evaluation Date

February 27, 2026 at the beginning of your scheduled lab.

1.2 Evaluation Method

Handwritten programming quiz where you will need to recount in pseudo-code aspects of this programming homework. The manual programming tasks will be relatively simple, but they will require a student to have practiced the homework to immediately understand and answer the question. Students who have not practiced or at least studied this homework, will generally be guessing during the quiz and they will typically require a long time to complete the quiz. Quizzes are timed and that time is capped at 30 minutes for all students including DSS students as this quiz involves recitation of already learned and practiced concepts and is not asking for full creative thinking. *Someone who has done this work should be able to finish the associated programming quiz in less than 10 minutes. If you have a handwriting disability, please talk to the professor before the evaluation date.*

1.3 Original Grading Rubric

I leave this here as an artifact for your reference. This is how this homework was graded when we evaluated student implementations. A rubric was provided with each homework and as the homeworks progressed, the percentages would encompass more critical metrics.

Coding Exercise:

- 20% For a working set of programs, both unit tests and profiler
- 40% For sort implementations (10% for each sort)
- 20% For passing all unit tests (5% for each block)
- 10% For sufficient comments
- 10% For consistent coding style

Analysis Exercise:

- 20% For presenting supporting data clearly and comprehensively
- 20% For clear, well-reasoned, well-supported analysis of Selection Sort growth rates
- 20% For clear, well-reasoned, well-supported analysis of Bubble Sort growth rates

- 20% For clear, well-reasoned, well-supported analysis of Insertion Sort growth rates
- 20% For clear, well-reasoned, well-supported analysis of Quicksort growth rates

1.4 Comments

Again, I have provided relevant comments to document the files (a file header), the function signatures (function block comments), and inline comments in much of the unit test file. These comments are a model that you should follow in the future when developing files or functions on your own. This level of comments will be expected in future assignments.

You must substitute your name in the file header where indicated and provide any inline comments to document code that you contribute to your solution.

In subsequent assignments, comments in the framework will begin to be omitted and you will be expected to document the code yourself. Again, I encourage you to look closely at how files, functions, and implementations have been documented so far and to follow these examples as a model for future work.

1.5 Use of Imported Java Types

A goal of this class is to develop an understanding of how to build data structures. We will be developing our own classes for ArrayLists, LinkedLists, Stacks, Queues, BinaryTrees, etc. For this reason, you are not permitted to import any of the existing classes from any library unless explicitly stated

2 A Class Schedule Model

Within each academic semester, classes must be scheduled within the school, the department, across departments, as well as scheduled such that students are able to take enough classes to meet degree requirements and graduate on time.

In order to optimize the classes a student is able to take, this schedule provides a wide variety of classes in many departments which all have a varying length of time. Sometimes classes are less than an hour, and in this case, classes range upwards of a few hours. With a given start time and end time, a student may calculate the duration of a class in order to estimate how many classes they can fit in within a certain period of time.

One major consideration of the duration of a class, usually combined with other external data (such as student enrollment, student feedback, and resulting grades) help to create a perspective for funding a department, accreditation for the university, or simply, the structure of the class between semesters.

Have you ever wondered how decisions are made for classes offered once a week for a few hours versus 2-3 times a week for a shorter duration?

- When dealing with sorting, there are two important considerations that must be balanced: how fast must the sort be and therefore how frequently the sort is performed, and how much memory is necessary to perform the sort. Which of these considerations is the most important requirement is a matter of analyzing the requirements of the product that you are developing.
- Within the context of a university, classes are continually changing, classrooms are constantly reserved, speed is important in order to gather feedback, and critical in order to find the key features to scale the university for more students.
- For a social media site with large servers, there is high frequency usage, speed dominates memory in requirements, but for other applications such as embedded code in a microcontroller where data update is infrequent but the resources are very limited, memory may be a much more significant concern.

2.1 Framework

A framework for a simple semester schedule has been provided which consists of the files `ClassSchedule.java`, `UnitTests.java`, `ProfileSorts.java`, `ScheduleReader.java`, and scheduling files `classes.csv`, `classes1k.csv`, and `classes10k.csv`. You will implement a number of methods in the `ClassSchedule` class which are validated by the `UnitTests` class and are profiled by the `ProfileSorts.java`. You must implement and validate all the methods in `ClassSchedule` before you can profile the sort performance. Additionally and as before, a `ClassSchedule.Framework.java` file is provided if you ever need to start over and a `ClassSchedule.Solution.java` file is provided with a demonstrable solution to the problem however you should not refer to this file unless you cannot program these algorithms yourself. You should not use generative AI to write any of the code in `ClassSchedule.java`; otherwise, you will have extreme difficulty on the quiz.

Remember that your submission must be comprehensive and include all files needed to compile and run your work. This means you need to include any assets, *i.e.* data files like the schedule files, needed to run your program.

We are asking that you implement a number of sorting algorithms. These algorithms are known and published, so you do not need to reinvent them, you just need to implement them with the restrictions created by the application. You may use the code on the course site as a reference (www2.seas.gwu.edu/jrt/cs1112/modules/module3/module3.html).

We often illustrate sorting using small arrays in class so that we can see sort behavior; however, the performance of sorts heavily depends on the size of the data and most realistic applications will involve large data sets. The `ScheduleReader.java` and `classes` files model and load a not small, *i.e.* ten thousand record, data set which is used for a test set. This data is large enough for us to start to see how each algorithm has unique performance characteristics. Once you have finished your implementation, you will use the `ProfileSorts` program to compare the performance of your sorts.

2.2 Data Structures

The framework assumes a number of array based data structures which are consistently named or prefixed as `schedule`, `view`, `classInfo`, and `profile`. This section specifies the intent and usage of the data structures.

2.2.1 `schedule`

`schedule` is a master list of class information. This data structure represents the current database of scheduling data for our semester set of classes. We should consider this the live, real-time data exported straight from the GWeb registration site. The data reflects the current state of the class information for the website at a single instant in time. Consider that this information may be changing drastically and you might grasp the scale of the problem of designing an efficient system that dictates the future of classes.

The structure of `schedule` consists of rows (1st dimension) of `class` records (2nd dimension). Each class record consists of five fields of Strings (see `schedule` below).

2.2.2 `classInfo`

A `classInfo` is a single record (or row) in a `schedule` of classes.

The structure of a `classInfo` consists of an array of five String values. The 1st field (index 0) is the class department which is usually a string of length 3 or 4. The 2nd field (index 1) is the class number which is a 4 digit number starting at 1000 represented as a string. The 3rd field (index 2) is the class title is the full class title for the class. The 4th field (index 3) is the start time is a string of the time the class starts provided as “HH:mm:ss” format; will always start before the end time. The 5th field (index 4) is the end time is a string of the time the class starts provided as “HH:mm:ss” format; will always be later than the start time.

Figure 1 illustrates both the `schedule` multidimensional array and the structure of each individual class.

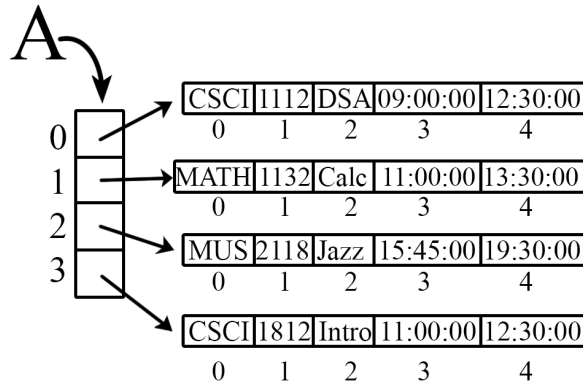


Figure 1: The above is representation of `schedule` data in memory where `A` is a reference to the data. Each “row” in the data is a single “class” which consists of five fields: class department, class number, class title, class start time, class end time. The class number index is a unique integer that is assigned when the class is submitted to the scheduling office with higher numbers representing classes most recently created, start time will always start before end time and both are given in 24hr time clocks.

2.2.3 view

A `view` is a referential (shallow) copy of the master list of `schedule`. A `view` is a shallow copy because it will allow changes to records in the master `schedule` to be reflected in the `view`. This shallow copy allows us to resequence the set of class data without actually manipulating the master `schedule`. We wish to use this extra layer because every user may have a unique `view` of the master data based on user preference and we wish to have a flexible way to manipulate user data without affecting the data of the master `schedule`.

The structure of a `view` is the same as the `schedule`.

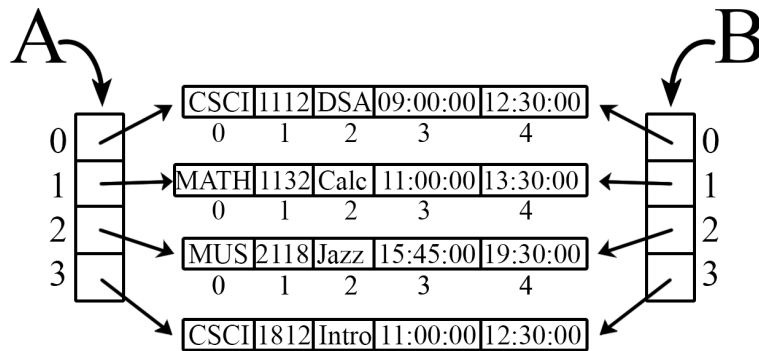


Figure 2: Let `B` be `view` data. When we create a `view`, we are creating a new array of references to the underlying rows in the `schedule` data. If we were to change a value in `view` or `schedule`, the changed value would be visible to both.

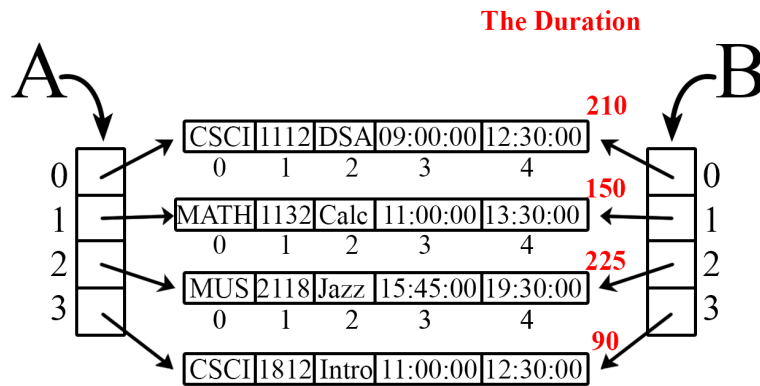


Figure 3: “The class duration” is a virtual field meaning that it is not directly represented in any of the class data. Instead, it is a dependent value that is calculated from the difference of a class’ start time and end time. The timing of classes are due to change rapidly, and by sorting by the duration of class, we risk it becoming stale data.

2.2.4 profile

A `profile` is an array that is used to track the performance of a sort.

The structure of a `profile` consists of an array of three integer values. The 1st field (index 0) will accumulate the number of allocations on the stack that have been made during the sort. The 2nd field (index 1) will accumulate the number of comparisons that are made during a sort. The 3rd field (index 2) will accumulate the number of swaps that are made during a sort.

It will be difficult to get a fully accurate measure for profile data and we really only want to get an idea of what the hidden costs are. You will not count any of the `profile` operations or allocations, but you will count these actions for the sort itself. When the profile data is printed for a full implementation, you should see orders of magnitude more allocs for recursive over iterative and orders of magnitude more compares and swaps for iterative over recursive which should help illustrate the hidden cost of these implementations

When profiling programs, we use a different approach; however, there is an important lesson about pass-by-value and changing data accessed by reference type hidden in this approach especially when implementing the recursive sorting.

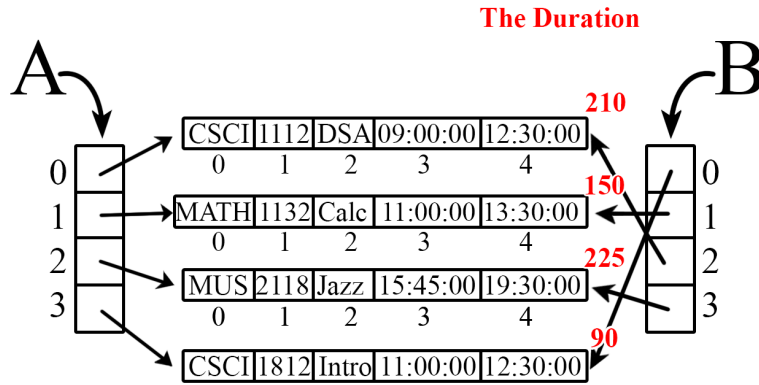


Figure 4: The view is sorted on the duration of classes while the schedule data is maintained in its original order of class number. With this model, we can maintain custom views of classes while also maintaining the schedule data in the order it was originally created and published.

3 Requirements

```

public static String[][] clone(String[][] schedule)
public static String[][] createView(String[][] schedule)
public static int differential(String[] classInfo)
private static int duration(String time)
public static boolean lessThan(String[] class1, String[] class2)
public static void swapClasses(String[][] view, int i, int j)
public static String[] sortSelection(String[][] view)
public static String[] sortBubble(String[][] view)
public static String[] sortInsertion(String[][] view)
public static String[] sortQuicksort(String[][] view)
public static int searchIterative(String[][] view, int diff)
public static int searchBinary(String[][] view, int diff)

```

3.1 Functions

This section gives a general explanation of the requirements for each function. More information may be available in the function headers and in how the functions are unit tested.

String[][] clone(String[][] schedule)

`clone` performs a deep copy or value level copy of a set of `schedule` and returns that copy. The intent is that we may periodically backup data from the data base and this type of operation creates a historical snapshot that will never change. If class times or information in the master set of `schedule` changes, we do not want the changes to be reflected in cloned data.

String[][] createView(String[][] schedule)

`createView` performs a shallow copy or referential copy of the rows in `schedule` data and returns this new array of row references. We will call an array of row references a “view”. The intent is that if class information in the master set of the `schedule` changes, we want the new information to be automatically known to the `view`. If we performed

a deep copy, we would need a separate and probably complicated process to synchronize an existing view with the new schedule data.

int differential(String[] classInfo)

`differential` computes the virtual differential—for a single `classInfo` record and returns the differential. The differential is simply the time difference between the start time and end time of a class in minutes. `differential` is virtual because it is a computed value from `classInfo` data that is not directly stored in a `classInfo` record.

int duration(String time)

`duration` is a utility function already provided that converts a time string in the format "HH:mm:ss" into an integer value representing minutes. This function will be useful when computing the `differential` as you will need to convert the start time and end times into numerical values in order to perform the arithmetic calculation.

boolean lessThan(String[] class1, String[] class2)

`lessThan` performs a less than comparison between two `class` records where `class1` acts as the left operand and `class2` acts as the right operand. The basis of comparison is the `differential` between the two operands. You must use the `differential` method in this implementation in your sorts. You must find a way to replace any `<`, `<=`, `>`, or `>=` operator for comparing two values in your sorts with this function.

void swapClasses(String[][] view, int i, int j)

`swapClasses` performs a shallow swap between two `class` records (at indices `i` and `j`) in a `view`. This is a referential swap and not a deep data swap. The intent is that the original `classInfo` data in the master set of schedule should be completely unaffected by this swapping operation.

int[] sortSelection(String[][] view)

`sortSelection` performs a selection sort on a `view` (a referential view of the master set of schedule). `sortSelection` returns an array of `profile` information as detailed in the Data Structures section above. You may implement additional functions to support this sorting operation; however, take care that the `profile` information is accurately maintained and accounted for over any subsequent function calls. You are implementing the sort as published on the course website, but you will need to make some changes to make it work on `classInfo` data.

int[] sortBubble(String[][] view)

`sortBubble` performs a bubble sort on a `view` (a referential view of the master set of schedule). `sortBubble` returns an array of `profile` information as detailed in the Data Structures section above. You may implement additional functions to support this sorting operation; however, take care that the `profile` information is accurately maintained and accounted for over any subsequent function calls. You are implementing the sort as published on the course website, but you will need to make some changes to make it work on `classInfo` data.

int[] sortInsertion(String[][] view)

`sortInsertion` performs an insertion sort on a `view` (a referential view of the master set of schedule). `sortInsertion` returns an array of `profile` information as detailed in the Data Structures section above. You may implement additional functions to support this sorting operation; however, take care that the `profile` information is accurately maintained and accounted for over any subsequent function calls. You are implementing the sort as published on the course website, but you will need to make some changes to make it work on `classInfo` data.

```
int[] sortQuicksort(String[][] view)
```

`sortQuicksort` performs a quicksort sort on a `view` (a referential view of the master set of `schedule`). `sortQuicksort` returns an array of `profile` information as detailed in the Data Structures section above. You may implement additional functions to support this sorting operation; however, take care that the `profile` information is accurately maintained and accounted for over any subsequent function calls. You are implementing the sort as published on the course website, but you will need to make some changes to make it work on `classInfo` data.

```
int searchIterative(String[][] view, int diff)
```

`searchIterative` performs a linear time complexity iterative search on a `view` (a referential view of the master set of `schedule`) searching for a calculated differential equal to the `diff` parameter. If a record in `view` is found with a differential matching the `diff` parameter, `searchIterative` returns a non-negative number representing the index of the record in `view`; otherwise, `searchIterative` returns -1.

```
int searchBinary(String[][] view, int diff)
```

`searchBinary` performs a logarithmic time complexity binary search on an ordered `view` (a referential view of the master set of `schedule`) searching for a calculated differential equal to the `diff` parameter. If a record in `view` is found with a differential matching the `diff` parameter, `searchBinary` returns a non-negative number representing the index of the record in `view`; otherwise, `searchBinary` returns -1.

4 Analysis

This section remains to give students context about the additional analysis that was original assignment. This was always a little flawed because the real measured numbers will always seemingly contradict a time complexity analysis, but this is to be expected since time complexity uses such broad estimation to arrive at a measurable quantity. This problem posed a challenge for students, and I was always trying to find the right way to get students to both implement and analyze their work. I leave this here as a reminder of these goals and expectations but with the admission that this exercise is a little wonky.

Along with your code, you must submit an analysis of the sorts in a .pdf file using the same naming conventions as the .zip file except with the .pdf extension. This .pdf must be included inside your .zip file. A simple way to write this file and generate a pdf of it is to write the file in google docs on your google drive and then print it which will download a pdf of your file from your google drive.

When you run the `ProfileSorts` application, you should see statistics similar to the following:


```

100 Classes
profileSelection::allocs:3, compares:4950, swaps:99
profileBubble::allocs:3, compares:4935, swaps:2949
profileInsertion::allocs:3, compares:2466, swaps:2375
profileQuicksort::allocs:808, compares:783, swaps:286

1000 Classes
profileSelection::allocs:3, compares:499500, swaps:999
profileBubble::allocs:3, compares:499490, swaps:313555
profileInsertion::allocs:3, compares:250009, swaps:249016
profileQuicksort::allocs:8224, compares:13984, swaps:7827

10000 Classes
profileSelection::allocs:3, compares:49995000, swaps:9999
profileBubble::allocs:3, compares:49995000, swaps:44522036
profileInsertion::allocs:3, compares:24985192, swaps:24975201
profileQuicksort::allocs:105160, compares:201583, swaps:75984

```

Your statistics may vary somewhat from the above sample depending on the way you implement your sorting algorithm; however, the general order of magnitudes for each of the statistics should be roughly the same. If your numbers are wildly different than the sample statistics, analyze the sample statistics in your pdf rather than your own data.

In your .pdf, paste the profile data that you are analyzing then write a paragraph for each sort explaining why that sort grows in the manner that it does with respect to the number of classes. For each sort, discuss the growth rates of the allocations, comparisons, and swaps with respect to the number of classes and argue why these numbers and growth rates are consistent with the specific strategies and overall performance approximation of the sorts in terms of memory and operations that we discussed in class. In other words, you are arguing whether or not the data supports the time complexity and space complexity estimates that we determined for each of these sorts.

According to the rubric, you will receive credit for “clear, well-reasoned, well-supported analysis of growth rates” for each of the sorting algorithms. What is a well-reasoned, well-supported analysis?

Well-supported simply means that your argument is based in the theory and evidence, in this case the profile data and the theory we applied in class to predict expected growth rates. While three samples is very small, it will be enough to confirm the expected growth rates as the input increases. Use the data as objective support for your analysis. An unsupported analysis is incomplete and cannot receive full credit.

Well-reasoned means that you never assume anything to be self-evident. Your argument must be consistent and link together your hypothesis with your conclusion based in objective assessment. For example, in class, we predicted that Selection Sort time complexity is approximately $O(n^2)$ and space complexity is approximately $O(1)$ where n is the size of the input (hypothesis). Based on the data generated by this experiment, does this hypothesis hold true (supports or refutes the hypothesis)? Objectively show how the trend in the data supports or refutes the hypothesis by analyzing the data and using this trend as the basis of your argument. Link your argument together to form a clear, cohesive argument that makes logical sense. Do NOT throw random thoughts (leaps in reasoning) together as it will undermine your argument. Propose an argument, show how the data supports or refutes your claim, formulate a conclusion based on what the data shows with respect to the argument.

While this may seem like a lot, we are really only looking for good paragraphs consisting of maybe 5 sentences. The number of sentences can vary greatly depending on your language skills; 3 or 4 well structured and well crafted sentences might do the trick where it may take 10 really simple sentences to achieve the same. When a scientist writes a paper, they write and rewrite these arguments over and over to communicate clearly and logically. A good process is to draft your arguments (you can even start now using our data) as bullet points, iterate over your statements to make

sure one leads to the next (like writing a program as one statement should logically lead to the next), join statements where appropriate with punctuation, conjunctions, clauses, etc. into a paragraph. Iterate over the paragraph to polish it.

Indicators of Generative AI in your analysis will result in a zero for the entire assignment. You must perform the analysis and make these arguments in your own words and without assistance from sources outside the class. This is one of the most important skills we are working on in this class.